

Question 1 :-

Write algorithms to compute factorial using both iterative & recursive methods & analyze their complexity.

- Write an iterative algorithm for factorial of n .
- Write an recursive algorithm for factorial of n .
- perform time & space complexity analysis under RAM model.
- compare iterative vs recursive approach.

→ Introduction :-

factorial of a number is the product of all positive integers less than or equal to that number. It is widely used in mathematics, probability and algorithm design. It can be calculated using both iterative & recursive methods.

Objectives :-

- To implement factorial using iterative & recursive methods.
- To analyze their time & space complexity.
- To compare both approaches.

Theory :-

factorial of a number n is defined as:

$$n! = n \times (n-1) \times (n-2) \times \dots \times 1$$

Base case:

$$0! = 1$$

$$1! = 1$$

There are two approaches:

1. Iterative : uses loops
2. Recursive : function calls itself

Algorithms :a. Iterative Algorithm :

- Start
- Read n
- Set $fact = 1$
- for $i = 1$ to n
 $fact = fact \times i$
- Print $fact$
- Stop

b) Recursive factorial algorithm

- i. start
- ii. If $n = 0$ or $n = 1$ then return 1
- iii. Else return $n \times \text{factorial}(n-1)$
- iv. Stop

Source code :-

```
#include <stdio.h>
int factorial_recur(int n) {
    if (n == 0 || n == 1)
        return 1;
    else:
        return n * factorial_recur(n-1);
}

int main() {
    int n, i;
    long long fact = 1;
    printf("Enter a number: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        fact *= i;
    }
    printf("Iterative factorial: %.11d\n", fact);
    printf("Recursive factorial: %.d\n", factorial_recur(n));
    return 0;
}
```

• complexity Analysis (RAM model)

method	Time complexity	Space complexity
Iterative	$O(n)$	$O(1)$
Recursive	$O(n)$	$O(n)$ (due to call stack)

Output :

Enter a number : 5

Iterative factorial : 120

Recursive factorial : 120

Conclusion :

- Both methods give same result.
- Iterative method / approach is more memory efficient.
- Recursive approach is easier to understand but uses more space.

Reference :

Introduction to Algorithm - Cormen
DSA text books.

Question 2:

Understand and apply Big-O, Big- Ω and Big Θ notation with graphical interpretation. Consider the following function:

$$f(n) = 3n^2 + 5n + 10$$

Plot geometrical interpretation for Big-O, Big- Ω , Big- Θ curves.
Write a C program with nested loops and determine its asymptotic complexity.

→ Introduction :-

Asymptotic notation are used to describe algorithm's efficiency as input size increases.

Objectives :-

- Understand Big-O, Big- Ω , Big Θ
- Graphically Interpret the function.
- Analyze a nested loop program.

Theory :-

Given,

$$f(n) = 3n^2 + 5n + 10$$

As n become larger

Dominant term = n^2

So:

$$\text{Big-O} = O(n^2)$$

$$\text{Big-}\Omega = \Omega(n^2)$$

$$\text{Big-}\Theta = \Theta(n^2)$$

Geometric Interpretation:

- Big O : Upper bound curve
- Big Ω : Lower bound curve
- Big Θ : Tight bound.

Graphical Interpretation:

First we identify the dominant term in $f(n) = 3n^2 + 5n + 10$. Since, n^2 grows faster than n or a constant, the complexity is quadratic. We use $g(n) = n^2$ as our reference function for all three notation.

for Big-O notation, we seek a constant c_2 and n_0 such that $f(n) \leq c_2 \cdot g(n)$, we can pick $c_2 = 8$ because for $n \geq 4$

$$3n^2 + 5n + 10 \leq 3n^2 + 5n^2 + 10n^2 = 18n^2$$

However, a tighter $c_2 = 8$ works efficiently for the plot as $3n^2 + 5n + 10 \leq 8n^2$ simplifies to $5n^2 - 5n - 10 \geq 0$, which holds for $n \geq 2$.

for Big-Omega notations, we seek a constant c_1 and n_0 such that $f(n) \geq c_1 \cdot g(n)$. Since, $3n^2$ is always less than

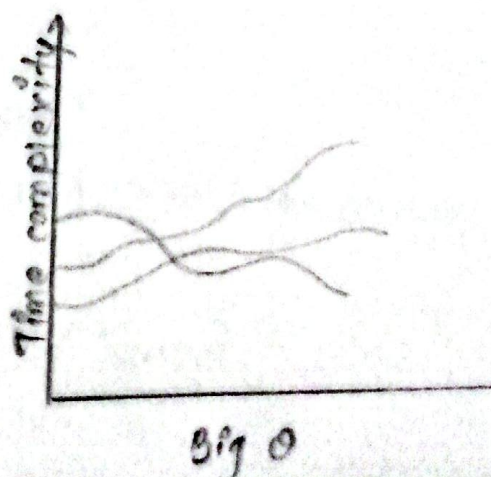
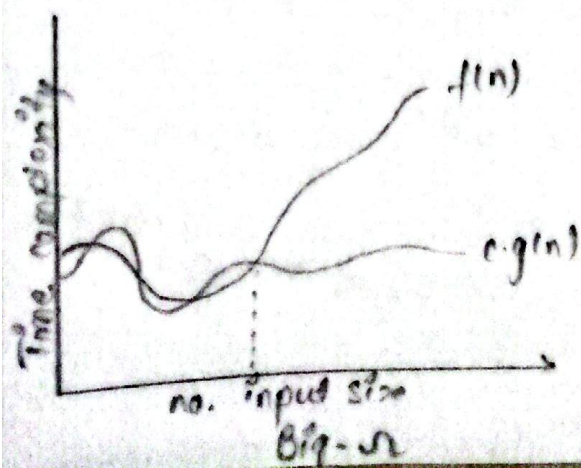
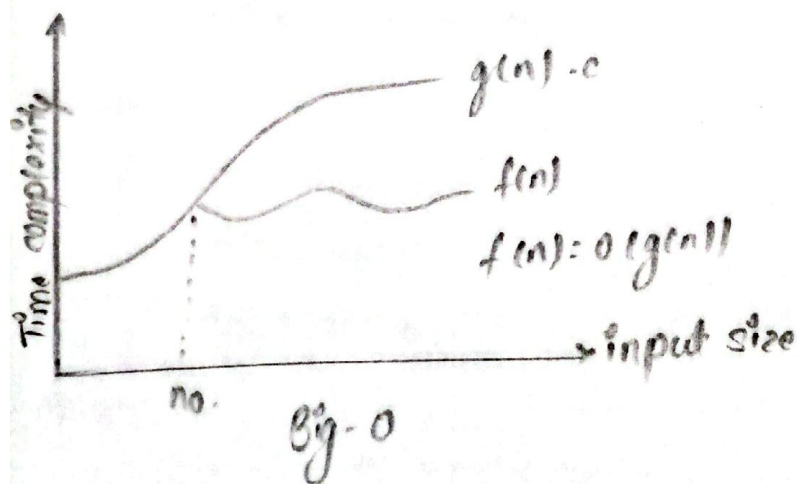
$3n^2 + 5n + 10$ for $n > 0$, we can safely choose $c_1 = 2$

$$2n^2 \leq 3n^2 + 5n + 10$$

This inequality holds for all $n \geq 1$.

Geometrical Representation:

- Big-O: The curve $8n^2$ stays above $f(n)$.
- Big- Ω : The curve $2n^2$ stays below $f(n)$.
- Big- Θ : $f(n)$ resides between $2n^2$ and $8n^2$ for all n beyond the critical point n_0 .



Source code :

```
#include <stdio.h>
int main() {
    int n, i, j;
    printf("Enter n: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            printf("Hello");
        }
        printf("\n");
    }
    return 0;
}
```

complexity

Outer loop : n

Inner loop : n

Total operations : $n \times n = n^2$

Time complexity = $O(n^2)$

Space complexity = $O(1)$

Output :-

For $n = 2$:

Hello Hello

Hello Hello

conclusion :

- Big - O gives worst case.
- Big - ω gives best case
- Big - Θ gives exact growth rate
- Nested loops usually produce quadratic complexity.

Reference :

- Algorithm Design :- Kleinberg & Tardos

Question 3:

Solve recurrence from recursive algorithms using different method. Give recurrence:

$$T(n) = 2T(n/2) + n$$

Implement recursive merge sort & verify recurrence complexity.

→ Introduction:-

Recursive algorithms are analyzed using recurrence relations.

Objectives :-

- Solve recurrence using different methods.
- Implement merge sort
- Verify complexity

Theory :

Given,

$$T(n) = 2T(n/2) + n$$

Using master theorem,

$$a = 2, b = 2, f(n) = n$$

$$n^{(\log_2 a)} = n^{(\log_2 2)} = n^1 = n$$

case 2 :

$$T(n) = O(n \log n)$$

Algorithm : merge sort .

- i. If array size = 1 → return
- ii. Divide array into two halves.
- iii. Recursively sort both halves.
- iv. merge the sorted halves.

Source code :

```
#include <stdio.h>
```

```
void merge (int arr[], int l, int m, int n) {
```

```
    int i, j, k;
```

```
    int n1 = m - l + 1;
```

```
    int n2 = n - m;
```

```
    int l1[n1], l2[n2];
```

```

for (i = 0; i < n1; i++)
    L[i] = arr[l + i];
for (j = 0; j < n2; j++)
    R[j] = arr[m + 1 + j];
i = j = 0;
k = l;
while (i < n1 && j < n2) {
    if (L[i] <= R[j])
        arr[k++] = L[i++];
    else
        arr[k++] = R[j++];
}
while (i < n1)
    arr[k++] = L[i++];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        mergeSort(arr, l, m, r);
    }
}

int main() {
    int arr[] = {5, 2, 4, 1, 3};
    int n = 5, i;
    mergeSort(arr, 0, n - 1);
    printf("sorted array: \n");
    for (i = 0; i < n; i++)
        printf("%d", arr[i]);
    return 0;
}

```

Conclusion :-

- Recurrence solved using master Theorem
- merge sort has time complexity $O(n \log n)$.
- Efficient for large datasets.

Reference

- CLRS Algorithm Book.

Question 4:

Implement basic iterative algorithms & analyze complexity.
Write an int iterative program for:

Write an iterative program to compute.

Analyze time & space complexity of both algorithms.

→ Introduction:

Iterative algorithms use loops instead of recursion.

Objective:

- Implement GCD and Fibonacci iteratively.
- Analyze complexity.

Theory:

GCD (Euclidean Algorithm)

Repeatedly replace:

$$a = b$$

$$b = a \% b$$

$$\text{Until } b = 0$$

Fibonacci

$$f(n) = f(n-1) + f(n-2)$$

Algorithm

GCD

i. Read a, b

ii. while $b \neq 0$

$$\text{temp} = b$$

$$b = a \% b$$

$$a = \text{temp}$$

iii. print a

Fibonacci

i. Read n

ii. Set $a = 0, b = 1$

iii. Repeat n times:

$$c = a + b$$

$$a = b$$

$$b = c$$

iv. Print sequence

Source code:

```
#include <stdio.h>
int main() {
    int a, b, temp, n, i, x = 0, y = 1, z;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    while (b != 0) {
        temp = b;
        b = a % b;
        a = temp;
    }
    printf("GCD: %d\n", a);
    printf("Enter n: ");
    scanf("%d", &n);
    printf("Fibonacci: ");
    for (i = 0; i <= n; i++) {
        printf("%d ", x);
        z = x + y;
        x = y;
        y = z;
    }
    return 0;
}
```

complexity

Algorithm	Time	Space
GCD	$O(\log \min(a, b))$	$O(1)$
Fibonacci	$O(n)$	$O(1)$

Output:

Enter two number: 12 18

GCD: 6

Enter n: 5

Fibonacci: 0 1 1 2 3

Conclusion :

- Iterative GCD is very efficient.
- Iterative fibonacci avoids recursion overhead.

Reference :

- Data structure by weiss.

Question 5 :-

Implement sequential search & three sorting algorithm with complexity comparison. Implement in c and analyze.

- Best case
- worst case
- Average case

Implement the following sorting algorithm :

- Bubble sort
- Selection sort
- Insertion sort

For each sort :

- count number of comparisons / swaps
- Analyze time complexity

compare performance for:

- Sorted array
- Reverse array
- Random array

→ Introduction:

Searching & sorting are fundamental operations in algorithms.

Objectives :

- Implement linear search.
- Implement Bubble, selection & Insertion sort.
- compare their complexities.

Theory :

Linear search : Search element one by one.

Time complexity :

- Best : $O(1)$
- worst : $O(n)$
- Average : $O(n)$

Algorithm:

Linear search

- i. Read array & key
- ii. For each element
- iii. If match $\rightarrow$ print position

Bubble sort:

- i. Compare adjacent elements.
- ii. Swap if out of order
- iii. Repeat until sorted.

Selection sort:

- i. find element which is smallest.
- ii. swap with first position.
- iii. Repeat.

Insertion sort:

- i. Take element
- ii. Insert into sorted part

Source code:

```
#include <stdio.h>

void bubblesort(int arr[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
```

```
void selection sort(int arr[], int n) {
    int i, j, min, temp;
    for (i = 0; i < n - 1; i++) {
        min = i;
```

14

```

for (j = i+1; j < n; j++) {
    if (arr[j] < arr[min])
        min = j;
    }
    temp = arr[i];
    arr[i] = arr[min];
    arr[min] = temp;
    }
}

void insertion_sort(int arr[], int n) {
    int i, key;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i-1;
        while (j >= 0 && arr[j] > key) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
}

int main() {
    int arr[] = {5, 2, 9, 1, 34};
    int n = 5;
    bubble_sort(arr, n);
    printf("sorted array: \n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}

```

Complexity comparison:

Algorithm	Best	Average	Worst
Linear search	$O(1)$	$O(n)$	$O(n)$
Bubble search	$O(n)$	$O(n^2)$	$O(n^2)$
Selection search	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion search	$O(n)$	$O(n^2)$	$O(n^2)$

Output :

Sorted array :

1 2 3 5 9

Conclusion :

Linear search is simple but slow for large data.

Bubble & selection sort are inefficient.

Insertion sort performs better on nearly sorted data.

Reference :

Introduction to Algorithms - Cormen
Data Structure Textbooks.